

Kubernetes VERDICT-7™ Troubleshooting Framework

InfraThrone Elite | Canonical Debugging Model for Kubernetes Outages

1. Executive Summary

Kubernetes is not one system; it's seven. Every production outage you've faced can be mapped to one of these subsystems. Without structure, war rooms fall into chaos: random log-grepping, dashboard-hopping, and "try a rollout" guesses.

The **VERDICT-7™ Framework** is our structured antidote.

It defines **seven pillars** of Kubernetes truth, each with **invariants**, **fast probes**, **deep dives**, and **clean exit criteria**.

In a war room, VERDICT-7™ keeps everyone aligned:

- The **Incident Commander (IC)** announces which pillar is being validated.
- Engineers run **fast probes** (90s). If broken, go deeper; if green, move on.
- This creates a **shared language**: "We're green at R, move to D."

The result: **evidence-driven debugging**, not chaos.

THE DEVOPS WAR ROOM

2. The VERDICT-7™ Pillars

V — Versions & Invariants

This pillar validates that **all version contracts and system invariants** across the Kubernetes stack are **within supported bounds and internally consistent**.

Think of this as the “**physics**” of the cluster — if your gravity constant is wrong, no amount of rocket debugging will help.

1. What “Versions & Invariants” Cover

- **Kubernetes version skew rules**
 - Control-plane components (apiserver, controller-manager, scheduler) must be compatible with each other.
 - Kubelets must not drift beyond supported skew (usually $n, n-1$).
- **CRDs (CustomResourceDefinitions)**
 - CRD schema must have **consistent storedVersions** across the cluster.
 - Old/deprecated versions (v1beta1, v1alpha1) must be migrated before upgrades.
- **Container runtimes**
 - Docker, containerd, CRI-O — must match the expected CRI API version.
 - Runtime config must align with Kubelet (e.g., cgroup driver).
- **Linux & Cgroup contracts**
 - Kubelet and runtime must agree on cgroup driver (cgroupfs vs systemd).
 - Kernel features (iptables/ipvs, overlayfs, seccomp, AppArmor) must be available.
- **Feature gates & API invariants**
 - New feature gates enabled in apiserver must also be supported by kubelets.
 - Removed APIs (e.g., Ingress v1beta1 → Ingress v1) must not be in use.

2. Why Failures Here Are Dangerous

- **Illusions everywhere:** If apiservers or kubelets are on unsupported versions, they may silently reject or mutate objects.
- **CRD split-brain:** Some CRDs may only be served in v1, while others are still stored in v1beta1 → cluster-wide schema chaos.
- **Runtime mismatches:** If kubelet expects systemd cgroups but containerd uses cgroupfs, pods will never become Ready.
- **Hidden bombs:** These don't show up until upgrades, reboots, or scale events.

3. Invariants (Must Always Hold True)

1. **Control-plane skew invariant**
 - Apiserver ≤ 1 minor version apart from controller-manager, scheduler, etc.
2. **Node skew invariant**
 - Kubelet ≤ 1 minor version behind apiserver.
3. **Runtime invariant**
 - Runtime CRI version matches kubelet expectation.
 - Cgroup driver matches (systemd vs cgroupfs).
4. **CRD invariant**
 - All CRDs have a single valid storedVersion.
5. **API invariant**
 - No object depends on a removed API version.

4. Fast Probes (90-Second Checks)

```
# Cluster & node versions
kubectl version --short
kubectl get nodes -o custom-columns=NAME:.metadata.name,VERSION:.status.nodeInfo.kubeletVersion

# Control-plane images
kubectl -n kube-system get pods -l component=kube-apiserver -o jsonpath='{.items[*].spec.containers[*].image}'

# CRD storedVersions
kubectl get crd -o json | jq -r '.items[] | .metadata.name + " -> " + (.status.storedVersions|join(","))'

# Container runtime & cgroup mode
crictl info | jq '.config.containerd.runtimes'
stat -fc %T /sys/fs/cgroup # reports cgroup version
```

5. Deep Dives

- **Feature gate drift:** `kubectl -n kube-system logs -l component=kube-apiserver | grep "feature-gates"`

Compare with kubelet flags on nodes.

- **Deprecated API usage:** `kubectl get events --all-namespaces | grep "deprecated"`
- **CRD schema validation:** `kubectl get crd <name> -o yaml | grep storedVersions`

If multiple versions show, you must migrate.

- **Runtime misalignment**
On node: `journalctl -u kubelet | grep cgroup`

6. Failure Signatures

- **Mixed versions:**
 - Apiserver at 1.27, kubelet at 1.24 → unsupported.
- **CRD chaos:**
 - One CRD stored in v1beta1, another only served as v1.
- **Runtime mismatches:**
 - Kubelet error: failed to run Kubelet: misconfiguration: kubelet cgroup driver: systemd is different from container runtime: cgroupfs.
- **Feature gate gaps:**
 - Apiserver rejects PodSecurity objects, kubelet still enforcing.

7. Clean Exit Criteria

You cannot mark **V** green until:

- All apiservers & kubelets are within the supported skew.
 - CRDs reconciled to one stored version.
 - Runtime/cgroup alignment confirmed.
 - No deprecated APIs in use.
 - Feature gates are aligned cluster-wide.
-

8. RCA Value of V

In the **Split-Brain Apocalypse** incident:

- Some CRDs were still in v1beta1 while 1.27 removed support.
 - This broke admission flows and contributed to divergent truths across apiservers.
 - Had **V** been checked **before** upgrade, the incident would've been prevented.
-

TL;DR

- **V = Versions & Invariants is the gatekeeper.**
 - If V is broken, everything else is noise.
 - Always validate skew, CRDs, runtimes, and feature gates *before* touching apiservers.
-

E — Entry & Truth

Definition: This pillar validates the **front door of Kubernetes (apiserver)** and its **source of truth (etcd)**.

If this layer is inconsistent, every other signal (pods, controllers, metrics) is untrustworthy.

Think of it as:

- **Entry:** Can clients consistently *enter* the cluster via apiservers?
- **Truth:** Do all apiservers agree on the *truth* (etcd state)?

If **E** fails → debugging downstream (pods, services, controllers) is meaningless because the **ground truth itself is fractured**.

1. What “Entry & Truth” Covers

- **API server consistency**
 - All apiservers must serve the same object state for the same resource.
 - ResourceVersion (RV) numbers must be monotonic across apiservers.
 - **etcd health & quorum**
 - etcd must have quorum (majority healthy).
 - No alarms (NOSPACE, CORRUPTION, LEADER_CHANGE storms).
 - Writes must commit within expected latency.
 - **Admission path**
 - Admission webhooks (validating/mutating) must respond within timeout.
 - Failing or unreachable webhooks can block all writes.
-

2. Why Failures Here Are Critical

- **Split-brain:** Different apiservers returning different ResourceVersions → deployments “exist” on one API endpoint and not the other.
- **etcd corruption:** Even if apiservers are up, stale reads or quorum loss cause status vs spec divergence.
- **Admission deadlocks:** A misconfigured or down webhook turns the API into a black hole.
- **Symptom amplification:** Everything downstream (controllers, scheduler, kubelet) consumes garbage data.

This is why in the **Split-Brain Apocalypse**, the first anomaly was: `kubectl get pods` → different results on different retries, depending on apiserver hit.

3. Invariants (Must Always Hold True)

1. API consistency invariant

- All apiservers return identical state for a given resource.
- ResourceVersion must be monotonic (never goes backwards).

2. etcd quorum invariant

- Majority of etcd members healthy, leader stable.
- No active alarms.

3. Admission invariant

- Webhooks respond within configured timeout.
 - Timeout policy (Ignore vs Fail) must not block cluster.
-

4. Fast Probes (90-Second Checks)

```
# Check all apiservers agree (using Kubernetes Service endpoints)
for h in $(kubectl -n kube-system get ep kubernetes -o jsonpath='{.subsets[*].addresses[*].ip}'); do
  KUBERNETES_SERVICE_HOST=$h kubectl --insecure-skip-tls-verify=true get ns default -o json \
  | jq -r "$h + \" rv=\" + .metadata.resourceVersion"
done

# etcd quorum & health (run on control-plane or with admin certs)
ETCDCTL_API=3 etcdctl endpoint status --write-out=table
ETCDCTL_API=3 etcdctl alarm list

# Admission latency in apiserver logs
kubectl -n kube-system logs -l component=kube-apiserver --since=5m | egrep -i "timeout|webhook|etcdserver"
```

If you see divergent RVs or etcd alarms → stop here. Don't move to R/D/I until E is stable.

5. Deep Dives

- **API Diffing**

```
# Compare state across apiservers for a Deployment
for h in $(kubectl -n kube-system get ep kubernetes -o jsonpath='{.subsets[*].addresses[*].ip}'); do
  KUBERNETES_SERVICE_HOST=$h kubectl --insecure-skip-tls-verify=true get deploy myapp -o yaml > /tmp/deploy-$h.yaml
done
diff /tmp/deploy-<ip1>.yaml /tmp/deploy-<ip2>.yaml
```

- **etcd latency histograms**

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --write-out=json endpoint status | jq '.[].status.dbSize'
```

- **Admission analysis**

```
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations
kubectl -n kube-system logs -l component=kube-apiserver | grep "admission"
```

- **Leader election churn**

```
ETCDCTL_API=3 etcdctl endpoint status --write-out=table | grep leader
```


6. Failure Signatures

- **Split-brain APIs**
 - ResourceVersion 10384729 on apiserver-A, 10384751 on apiserver-B.
 - **etcd instability**
 - LEADER_CHANGE count spikes.
 - Alarms: NOSPACE, CORRUPTION.
 - **Admission stalls**
 - Apiserver logs: Timeout: request did not complete within requested timeout.
 - All create/update requests hang.
-

7. Clean Exit Criteria

You cannot mark **E** green until:

- All apiservers return the same state for the same resource.
 - ResourceVersions are monotonic across retries.
 - etcd quorum stable (leader fixed, no alarms).
 - Admission webhooks reachable, latency < timeout threshold.
-

8. RCA Value of E

In the **Split-Brain Apocalypse**:

- Apiservers diverged due to etcd leader churn.
- CRD schema mismatches compounded divergence.
- Admission webhooks started timing out, blocking pod creation.

Debugging scheduler (D) or kubelet (I) without fixing **E** would have been wasted effort.

9. War-Room Checklist for E

1. Verify apiservers return consistent ResourceVersions.
 2. Verify etcd quorum (endpoint status, alarms).
 3. Check admission webhooks for timeouts.
 4. Check apiserver logs for latency/corruption signs.
 5. Declare **E = Green** only if all above hold.
-

TL;DR

- **E = Entry & Truth is the heartbeat of the control plane.**
 - If apiservers don't agree, or etcd is unstable, you're not debugging workloads — you're debugging lies.
 - Fix E first, or everything else is noise.
-

THE DEVOPS WAR ROOM

R — Reconciliation

Definition: Reconciliation is the heartbeat of Kubernetes: controllers continuously compare the **desired state (spec)** with the **observed state (status)**, and act to align them.

If **R** breaks, the control plane stops “steering the ship.” Deployments don’t roll out, Jobs never complete, and objects rot in drift.

Think of **R** as the **autopilot of the cluster** — without it, you’re flying blind.

1. What “Reconciliation” Covers

- **Controller-manager health**
 - Deployment, ReplicaSet, StatefulSet, DaemonSet, Job controllers.
 - Node and endpoint controllers.
 - **Scheduler control-loop inputs**
 - Even before placement, controllers create or update pods that the scheduler consumes.
 - **Leader election**
 - Only one controller-manager and one scheduler should be active at a time.
 - **Event pipeline**
 - Events must be continuously generated and updated to reflect transitions.
 - **Workqueue health**
 - Controllers use workqueues; if stuck, reconciliation halts silently.
-

2. Why Failures Here Are Critical

- **Frozen workloads:** ReplicaSets stop creating pods → “Pending 0/0 created.”
- **Zombie specs:** Status never updates, even though underlying objects change.
- **Leader churn:** Constant flipping → no controller maintains steady progress.

- **Silent stalls:** Workqueues can silently back up; no pods, no events, no errors.

In the **Split-Brain Apocalypse**, even after stabilizing **E (Entry & Truth)**, controllers could not reconcile because leader election objects were stale.

3. Invariants (Must Always Hold True)

1. Stable leaders

- One controller-manager leader, one scheduler leader.

2. Spec → status convergence

- When spec changes, status should update within seconds to minutes.

3. Event stream flow

- Events must continuously reflect object state transitions.

4. Workqueue health

- Queue depth grows/shrinks predictably; not permanently stuck.

THE DEVOPS WAR ROOM

4. Fast Probes (90-Second Checks)

```
# Who are the leaders?
kubectl -n kube-system get lease | egrep 'controller|scheduler'

# Events flowing?
kubectl get events -A --sort-by=.lastTimestamp | tail -n 20

# Controller-manager logs
kubectl -n kube-system logs -l k8s-app=kube-controller-manager --since=5m | \
  egrep -i "error|lead|workqueue"
```

If leaders are flipping or events are stalled, stop here and fix **R** before moving on.

5. Deep Dives

- **Check workqueue metrics**

Port-forward controller-manager metrics:

```
kubectl -n kube-system port-forward deploy/kube-controller-manager 10257:10257  
curl -s localhost:10257/metrics | egrep "workqueue_depth|queue_duration"
```

Look for queues that are consistently non-zero with no drain.

- **ReplicaSet sanity:** `kubectl describe rs -n <ns> <name> | egrep "Desired|Current|Ready|Events"`
 - **Leader election churn details:** `kubectl get lease -n kube-system kube-controller-manager -o yaml`
 - If `renewTime` is bouncing across controllers, leadership is unstable.
-

6. Failure Signatures

- **Leader flip storms:**
 - Lease object changes owners every few seconds.
 - Controllers restart loops endlessly.
 - **Silent freeze:**
 - `kubectl apply -f deploy.yaml` returns success, but no pods appear.
 - Events show nothing after spec change.
 - **Workqueue backlog:**
 - Metrics show thousands of queued items, never draining.
 - **Partial reconciliation:**
 - Some Deployments reconcile, others don't, usually correlated with stale ResourceVersions from E.
-

7. Clean Exit Criteria

You cannot mark **R** green until:

- One stable leader for controller-manager and scheduler.
 - Events are actively flowing cluster-wide.
 - Workqueue depths are draining normally.
 - Spec → status transitions complete within expected time.
-

8. RCA Value of R

In the **Split-Brain Apocalypse**:

- etcd churn (E pillar) caused **controller-manager leader lease flapping**.
 - Workqueues stalled, leading to Deployments with **desired=5, current=0** stuck indefinitely.
 - Debugging scheduler or kubelet would have been pointless until controller reconciliation was restored.
-

9. War-Room Checklist for R

1. Check controller-manager & scheduler leaders.
 2. Verify events are flowing.
 3. Inspect ReplicaSets/Jobs for spec ≠ status anomalies.
 4. Check workqueue depth metrics.
 5. Declare **R = Green** only when spec → status convergence is observed.
-

TL;DR

- **R = Reconciliation is the autopilot.**
- If leaders flip, events stall, or queues back up, your cluster is “flying manual” with no corrections.
- Fix R before debugging scheduling or kubelet instantiation.

D — Decision & Placement

Definition: This pillar validates the **scheduler's decision-making pipeline**:

- **Filter** → which nodes *can* host a pod (taints, affinity, resources).
- **Score** → which node is *best*.
- **Bind** → set nodeName and commit the decision to the apiserver.

If **D** breaks, pods remain **Pending**, preemptions storm, or workloads land on nodes that collapse under pressure.

1. What “Decision & Placement” Covers

- **Scheduler health & leadership**
 - Only one active scheduler leader must exist.
- **Filtering**
 - Node taints/tolerations, resource availability, affinity/anti-affinity rules.
- **Scoring**
 - CPU/memory balancing, topology spread, custom plugins.
- **Binding**
 - Pod object updated with nodeName.
 - Binding must be reflected in etcd.
- **Preemption & fairness**
 - Pods may evict lower-priority workloads if quotas allow.

2. Why Failures Here Are Critical

- **Pending storms:** Thousands of pods stuck with 0/... nodes available.
- **Unpredictable placement:** Scheduler places pods unevenly, overloading some nodes.
- **Preemption loops:** High-priority pods trigger constant evictions with no progress.
- **Binding blackholes:** Scheduler “decides” but apiserver never reflects the binding.

In the **Split-Brain Apocalypse**, stale truths from **E (Entry & Truth)** caused scheduler to read outdated node/pod info → pods sat in Pending despite available nodes.

3. Invariants (Must Always Hold True)

1. Leader invariant

- Only one kube-scheduler leader active.

2. Filter invariant

- At least one node passes filters for schedulable pods.

3. Score invariant

- Nodes are ranked deterministically.

4. Binding invariant

- Every scheduled pod has a nodeName assigned in spec.
-

4. Fast Probes (90-Second Checks)

```
# Pending pods snapshot
kubectl get pods -A --field-selector=status.phase=Pending -o wide | head

# Scheduler leader lease
kubectl -n kube-system get lease | grep scheduler

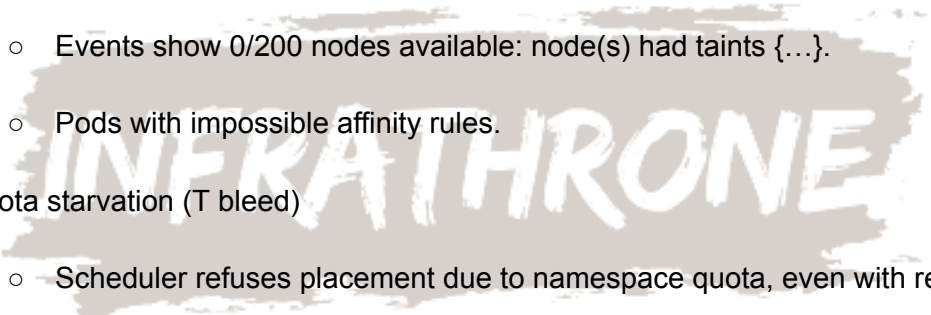
# Scheduler logs
kubectl -n kube-system logs -l component=kube-scheduler --since=5m | \
  egrep -i "FailedScheduling|error|preemption"
```


If Pending queue is large and logs show repeated 0/... nodes available, you must fix **D** before moving downstream.

5. Deep Dives

- **Inspect a Pending pod:** `kubectl -n <ns> describe pod <pod-name> | egrep -A4 "Events|Warning|FailedScheduling"`
- **Node sanity:** `kubectl get nodes -o json | jq -r '.items[] | [.metadata.name,.status.conditions[]|select(.type=="Ready")|.status] | @tsv'`
- **Resource accounting :** `kubectl describe node <node-name> | egrep "Allocatable|Allocated|Pods"`
- **Preemption activity:** `kubectl -n kube-system logs -l component=kube-scheduler --since=10m | grep Preemption`

6. Failure Signatures

- Filter bottleneck
 - Events show 0/200 nodes available: node(s) had taints {...}.
 - Pods with impossible affinity rules.
 - Quota starvation (T bleed)
 - Scheduler refuses placement due to namespace quota, even with resources.
 - Binding failures 

 - Pod spec never gets nodeName, even after scheduling attempt.
 - Preemption storms
 - Logs filled with preemption attempts but no successful bindings.
-

7. Clean Exit Criteria

You cannot mark D green until:

- Pending pods have either:
 - been scheduled to nodes, OR

- show explicit, explainable reasons (quota, taints, affinity).
 - One stable scheduler leader.
 - Binding pipeline confirmed (pods get nodeName).
 - Preemption, if occurring, results in progress, not infinite churn.
-

8. RCA Value of D

In the Split-Brain Apocalypse:

- Scheduler consumed stale API truth (from E) → thought nodes were unavailable.
 - Result: waves of FailedScheduling events with 0/... nodes available.
 - Real fix required stabilizing E, then validating R, before D returned to green.
-

9. War-Room Checklist for D

1. Check scheduler leader.
 2. Snapshot Pending pods.
 3. Inspect Events for FailedScheduling reasons.
 4. Validate node readiness & resources.
 5. Check for preemption loops.
 6. Declare D = Green only when pods are binding or failures are well explained.
-

TL;DR

- D = Decision & Placement is the brain of workload placement.
- If the scheduler is blind (due to stale truth, taints, or quotas), pods rot in Pending.
- Fix D before looking at kubelet instantiation or networking.

I — Instantiation

Definition: Instantiation is the process by which a **scheduled Pod** (with nodeName set by the scheduler) is:

1. Accepted by kubelet,
2. A sandbox created by the container runtime,
3. Networking attached via CNI,
4. Volumes mounted,
5. Images pulled and containers started,
6. Probes executed until Ready.

If I fail, pods remain in **ContainerCreating**, **CrashLoopBackOff**, or **NotReady** even though scheduling (D) succeeded.

1. What “Instantiation” Covers

- **Kubelet health** (node agent): pulls pod manifests, interacts with CRI, manages lifecycle.
- **Container runtime** (containerd, CRI-O, dockershim legacy).
- **CNI plugins** (flannel, calico, cilium, weave, etc.).
- **Volume management** (CSI plugins, hostPath, persistent disks).
- **Image pulls** (registries, credentials, caching).
- **Health checks** (liveness, readiness, startup probes).

2. Why Failures Here Are Critical

- Scheduler assigns pod → nothing starts.
- Pods stuck in **ContainerCreating** forever.

- CrashLoop storms drain nodes.
- Kubelet flaps between Ready/NotReady, destabilizing workloads.

In the **Split-Brain Apocalypse**, once apiserver truth (E) and scheduler (D) stabilized, kubelets still failed to instantiate pods due to **admission webhooks timeouts + CNI plugin failures** on some nodes.

3. Invariants (Must Always Hold True)

1. Kubelet invariant

- Kubelet is healthy and heartbeating with apiserver.

2. Runtime invariant

- Pod sandboxes can be created via CRI.

3. CNI invariant

- Pods get IPs and routes within network policies.

4. Volume invariant

- Mounts succeed for all declared volumes.

5. Image invariant

- Required images can be pulled (auth, size, cache).

6. Probe invariant

- Liveness/readiness/startup probes eventually pass.

4. Fast Probes (90-Second Checks)

```
# Pod stuck in ContainerCreating or CrashLoop?
kubectl get pods -A -o wide | egrep "ContainerCreating|CrashLoopBackOff" | head

# On a suspect node: kubelet logs
journalctl -u kubelet --since=10m | egrep -i "cni|sandbox|runtime|probe|image|mount"

# Runtime state
crictl ps -a | head
crictl inspectp <pod-id> | jq '.status, .info'

# Quick CNI test: launch a debug pod
kubectl run netdebug --image=busybox:stable --rm -it -- sh -c "ip a; ping -c1 kubernetes.default"
```

5. Deep Dives

- Image pull analysis

```
crictl images
kubectl describe pod <pod> | grep -A3 "Failed to pull image"
```

- CNI plugin sanity

```
ls /etc/cni/net.d
cat /etc/cni/net.d/*.conf
```

Check `/var/log/` or node syslog for CNI plugin errors.

- **Volume failures** : `kubectl describe pod <pod> | grep -A5 "MountVolume"`

Look for timeouts on CSI drivers (AWS EBS, GCE PD, etc.).

- **Probe failures** : `kubectl describe pod <pod> | grep -A3 "Liveness|Readiness"`

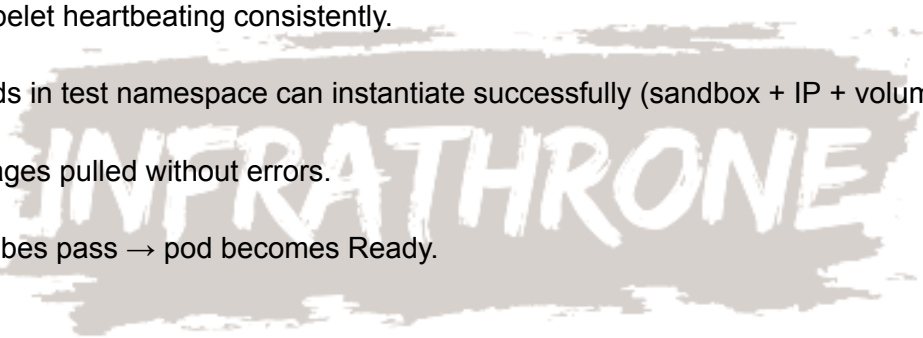
6. Failure Signatures

- Sandbox failures
 - FailedCreatePodSandbox in kubelet logs.
 - CRI errors like `rpc error: code = Unknown desc = failed to create containerd task`.
- CNI errors
 - Error adding network: failed to find plugin "calico" in path.
 - Pods show ContainerCreating indefinitely.
- Image pull issues

- ErrImagePull, ImagePullBackOff.
 - Registry auth expired, or huge images clogging pulls.
 - Volume mount issues
 - MountVolume.SetUp failed for volume "pvc-xxx".
 - Probe storms
 - Pods cycling due to failing readiness/liveness checks.
-

7. Clean Exit Criteria

You cannot mark I green until:

- Kubelet heartbeating consistently.
 - Pods in test namespace can instantiate successfully (sandbox + IP + volumes).
 - Images pulled without errors.
 - Probes pass → pod becomes Ready.
- 
-

THE DEVOPS WAR ROOM

8. RCA Value of I

In the Split-Brain Apocalypse:

- Admission webhooks (T pillar) timed out → kubelet failed to instantiate new pods.
 - On some nodes, CNI failed due to outdated config, leaving pods in ContainerCreating.
 - Resolution required aligning E → R → D, then fixing runtime/CNI configs to restore I.
-

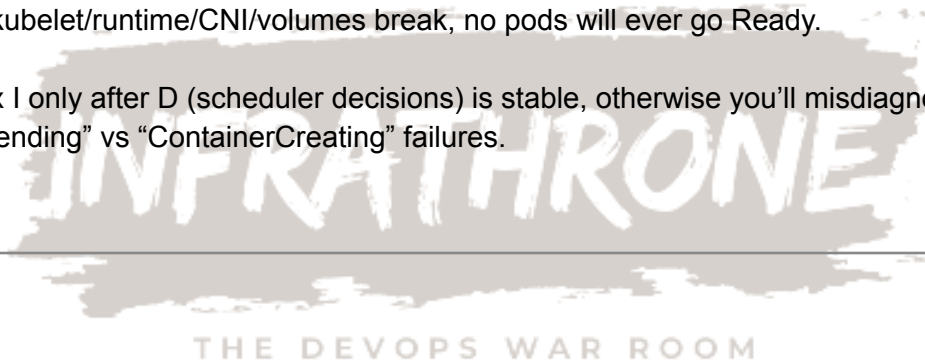
9. War-Room Checklist for I

1. Identify pods stuck in ContainerCreating or CrashLoopBackOff.

2. Inspect kubelet logs on failing nodes.
 3. Check CRI runtime state (crictl ps, crictl inspectp).
 4. Verify CNI plugin availability and logs.
 5. Validate image pulls and volume mounts.
 6. Run debug pod to confirm network and readiness.
 7. Declare I = Green only when pods instantiate cleanly across nodes.
-

TL;DR

- I = Instantiation is where intent becomes reality.
- If kubelet/runtime/CNI/volumes break, no pods will ever go Ready.
- Fix I only after D (scheduler decisions) is stable, otherwise you'll misdiagnose "Pending" vs "ContainerCreating" failures.



C — Connectivity & Discovery

Definition: Connectivity & Discovery ensures that workloads can **find and talk to each other** reliably:

- **Services & Endpoints:** mapping from stable service names → backend pods.
- **kube-proxy / iptables / IPVS:** actual packet routing rules.
- **CoreDNS:** service discovery via DNS.
- **Cluster networking (CNI, routes, kube-proxy sync).**
- **Ingress / Load Balancer path:** external traffic into cluster.

If **C** fails, pods may be **Ready**, but users get **502s, timeouts, or DNS errors**.

1. What “Connectivity & Discovery” Covers

- **Endpoint objects:** Services must have healthy pod IPs attached.
 - **DNS resolution:** CoreDNS must resolve *.svc.cluster.local.
 - **kube-proxy health:** iptables/IPVS rules must match Endpoints.
 - **L4/L7 Load Balancers:** External ingress → Service → Pod path must work.
 - **Network policy interaction:** Calico, Cilium, or other CNIs must allow expected traffic.
-

2. Why Failures Here Are Critical

- Pods appear healthy → users still see outages.
- “Blackhole” traffic: Services exist, but no Endpoints → ingress returns 502.
- kube-proxy drift → Endpoints exist in etcd, but not in node rules.
- DNS failures → pods can’t resolve services, cascading timeouts.

In the **Split-Brain Apocalypse**, ingress controllers returned **502s** even when pods were **Ready** — kube-proxy + DNS drift caused **C pillar collapse**.

3. Invariants (Must Always Hold True)

1. Endpoint invariant

- Every Service has matching Endpoints for Ready pods.

2. DNS invariant

- CoreDNS resolves `kubernetes.default.svc.cluster.local`.

3. Proxy invariant

- kube-proxy syncs Endpoints into iptables/IPVS rules.

4. Path invariant

- A golden request (Ingress → Service → PodIP) succeeds.

4. Fast Probes (90-Second Checks)

```
# Check endpoints
kubectl get svc -A
kubectl get endpoints -A | egrep -v '(<none>|0/0)'

# DNS resolution test
kubectl -n kube-system run -it dnstest --image=busybox:stable --rm -- nslookup kubernetes.default

# kube-proxy health
kubectl -n kube-system logs -l k8s-app=kube-proxy --since=5m | egrep -i "sync|error|iptables|ipvs"

# Golden path: curl a known service
kubectl run netprobe --rm -it --image=curlimages/curl -- \
  curl -sS http://kubernetes.default.svc.cluster.local
```

5. Deep Dives

- Endpoint vs Pod readiness mismatch

```
kubectl get endpoints <service> -o wide
kubectl describe pod <pod> | grep -A5 Conditions:
```

Sometimes pod is running but not Ready → excluded from Endpoints.

- kube-proxy sync mode

```
ps aux | grep kube-proxy
# Check if running in iptables or IPVS mode
```

- Node-level routing rules

```
iptables -L -t nat | grep <service-cluster-ip>
ipvsadm -ln | grep <service-cluster-ip>
```

- **CoreDNS health:** `kubectl -n kube-system logs -l k8s-app=coredns --since=5m | grep SERVFAIL`
- **Ingress / LB health**

```
kubectl describe ingress <name>
kubectl -n kube-system get pods -l app=ingress-nginx
```

6. Failure Signatures

- Empty Endpoints
 - `kubectl get endpoints mysvc → <none>`.
 - Root cause: pods not Ready, mislabelled selectors.
- DNS failures
 - CoreDNS logs filled with SERVFAIL or upstream timeout.
 - Pods show Temporary failure in name resolution.
- Proxy drift
 - Endpoints exist, but iptables/IPVS rules missing.
 - kube-proxy logs: `syncProxyRules failed`.

- 502s at ingress
 - Ingress controller can't resolve Service backends.
 - Often, a symptom of upstream DNS/Endpoints collapse.
-

7. Clean Exit Criteria

You cannot mark C green until:

- Services have Endpoints for Ready pods.
 - CoreDNS resolves `kubernetes.default` consistently.
 - kube-proxy sync loops show no errors.
 - A golden-path request (Ingress → Service → Pod) returns HTTP 200.
-

8. RCA Value of C

In the Split-Brain Apocalypse:

- Endpoints objects existed but kube-proxy rules lagged due to etcd split-brain.
 - CoreDNS intermittently failed → ingress controllers saw no backends.
 - External traffic collapsed into 502 storms, even though pods were healthy.
-

9. War-Room Checklist for C

1. Check Service → Endpoint mappings.
 2. Verify DNS resolution from inside cluster.
 3. Inspect kube-proxy logs for sync failures.
 4. Validate node iptables/IPVS rules.
 5. Confirm ingress/LB golden-path HTTP request works.
 6. Declare C = Green only when traffic flows end-to-end.
-

TL;DR

- C = Connectivity & Discovery is the wiring layer.
 - Pods can be Running, but users still see 502s if C is broken.
 - Always prove golden-path traffic works before closing incident.
-

T — Tenancy & Policy

Definition: Tenancy & Policy governs the **rules of engagement** inside a Kubernetes cluster.

It covers **authorization, quotas, policies, PodDisruptionBudgets, admission webhooks, and security baselines.**

If **T** fails, workloads may be blocked *not because of technical faults* but because rules say “No.”

1. What “Tenancy & Policy” Covers

- **RBAC (Role-Based Access Control)**
 - Who is allowed to create/update/delete resources.
- **Admission webhooks (validating/mutating)**
 - Custom gates before objects are persisted.
- **Quotas & Limits**
 - Namespace ResourceQuotas, LimitRanges.
- **PodDisruptionBudgets (PDBs)**
 - Restrict voluntary evictions.
- **NetworkPolicies / PodSecurity / OPA Gatekeeper**
 - Control east–west communication and pod-level security.
- **API Priority & Fairness (APF)**
 - Queueing and throttling of requests under load.

2. Why Failures Here Are Critical

- **Blocked workloads:** Pods fail to create with Forbidden or quota exceeded.
- **Cluster-wide deadlocks:** A broken admission webhook blocks *all* API writes.
- **Invisible throttling:** APF silently starves low-priority controllers.
- **Policy misfires:** NetworkPolicy blocks service discovery; PSA prevents pod startup.

In the **Split-Brain Apocalypse**, admission webhooks timing out compounded failures: kubelets couldn't instantiate pods (I pillar) because admission never returned.

3. Invariants (Must Always Hold True)

1. **Auth invariant**
 - AuthN/AuthZ (RBAC) allow legitimate operations.
 2. **Admission invariant**
 - All required webhooks reachable; timeouts don't block cluster.
 3. **Quota invariant**
 - ResourceQuotas and LimitRanges enforced predictably, not exceeded.
 4. **Policy invariant**
 - NetworkPolicies, PodSecurity, PDBs applied deliberately, not blocking expected workloads.
 5. **Fairness invariant**
 - API Priority & Fairness queues drain fairly under load.
-

4. Fast Probes (90-Second Checks)

```
# Can workloads be created?
kubectl auth can-i create pods --as=system:serviceaccount:default:sa -n testns

# Quota and limits
kubectl get resourcequota,limitrange -A

# PodDisruptionBudgets
kubectl get pdb -A

# Admission webhooks present?
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations

# Apiserver logs for admission
kubectl -n kube-system logs -l component=kube-apiserver --since=5m | grep admission
```

5. Deep Dives

- **Webhook sanity**

```
kubectl get validatingwebhookconfigurations <name> -o yaml
kubectl -n <ns> logs <webhook-pod>
```

Check TLS validity, DNS resolution, and service endpoints.

- **Quota enforcement:** `kubectl describe quota -n <ns>`

Compare used vs hard.

- **NetworkPolicy troubleshooting**

```
kubectl get networkpolicy -A
kubectl -n test run nettest --rm -it --image=busybox:stable -- sh -c "nc -vz <pod-ip> 80"
```

- **APF fairness queues**

```
kubectl get flowcontrol
kubectl get prioritylevelconfiguration
```

6. Failure Signatures

- Forbidden errors
 - Error from server (Forbidden): pods is forbidden: User "X" cannot create resource "pods".
- Webhook timeouts
 - Apiserver logs: Timeout: request did not complete within requested timeout.
 - Cluster appears frozen even though apiserver is “up.”
- Quota exceeded
 - Pod stuck: exceeded quota: cpu-requests.
- PDB violation
 - Evictions denied even though cluster autoscaler/upgrade runs.
- Network blackholes
 - Services exist but traffic blocked due to restrictive NetworkPolicy.

7. Clean Exit Criteria

You cannot mark T green until:

- Legitimate users/service accounts can create/update resources.
 - Admission webhooks responsive, < timeout, TLS/DNS correct.
 - ResourceQuotas and LimitRanges respected but not over-enforcing.
 - NetworkPolicies, PodSecurity, and PDBs verified not blocking golden path.
 - APF queues draining normally.
-

8. RCA Value of T

In the Split-Brain Apocalypse:

- Admission webhooks pointed to services unreachable due to C (Connectivity) issues.
 - Apiserver treated timeouts as Fail → blocked all pod creation.
 - Namespace quotas mismatched → schedulable pods rejected by admission.
 - Fixing T required patching webhook failurePolicy=Ignore temporarily to unblock cluster.
-

9. War-Room Checklist for T

1. Run kubectl auth can-i for failing workloads.
 2. Inspect webhook configurations & logs.
 3. Check quotas, limits, and PDBs for blocking conditions.
 4. Validate NetworkPolicy paths with debug pods.
 5. Confirm PodSecurity/OPA rules align with intent.
 6. Mark T = Green only when workloads are admitted without unintended denials.
-

TL;DR

- T = Tenancy & Policy is the governance layer.
 - If T fails, the cluster is “alive” but rules forbid progress.
 - Always validate webhooks, quotas, and RBAC before assuming infra breakage.
-

3. Mapping Symptoms → Pillars

Symptom	Pillar
Different kubectl results on retries	E
Pods Pending with 0/... nodes available	D/T
FailedCreatePodSandbox, CNI errors	I
Ingress 502 with Ready pods	C
Sudden Forbidden/Quota errors	T

4. War-Room Operating Model

1. IC announces pillar under investigation.
2. Engineers run **90s fast probes**.
3. If green → move on. If red → deep dive.
4. Exit criteria must be satisfied before pillar marked "green."
5. No skipping pillars; no parallel chaos.

5. Why VERDICT-7™ Works

- **Evidence-driven:** no guessing, every step is falsifiable.
 - **Scales:** applies to dev clusters and global fleets.
 - **Trainable:** juniors learn structure, seniors validate instinct.
 - **Universal language:** “We’re green at R, move to D.” reduces noise.
-

6. Golden Path Checklist (Quick Copy)

```
# V
kubectl version --short
kubectl get crd -o json | jq -r '.items[].status.storedVersions'

# E
etcdctl endpoint status
kubectl get ns default -o json | jq '.metadata.resourceVersion'

# R
kubectl -n kube-system get lease
kubectl get events -A | tail -n 20

# D
kubectl get pods -A --field-selector=status.phase=Pending -o wide

# I
journalctl -u kubelet --since=10m | tail -n 50

# C
kubectl run -it netprobe --image=curlimages/curl --rm -- \
  sh -lc "nslookup kubernetes.default"

# T
kubectl get validatingwebhookconfigurations
kubectl auth can-i create pods --as=system:serviceaccount:default:sa
```

Saurav Chaudhary

Senior DevOps Consultant • Infra Architect • Mentor • War Room Specialist